

Stratum V1, V2, and DATUM

Agenda

- Stratum V1 jobs and shares

Agenda

- Stratum V1 jobs and shares
- Who controls a V1 block template

Agenda

- Stratum V1 jobs and shares
- Who controls a V1 block template
- DATUM's gateway model

Agenda

- Stratum V1 jobs and shares
- Who controls a V1 block template
- DATUM's gateway model
- Stratum V2's three protocols

Agenda

- Stratum V1 jobs and shares
- Who controls a V1 block template
- DATUM's gateway model
- Stratum V2's three protocols

Stratum V1

Terms

- **Block template:** candidate block contents before proof of work

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** template-derived search space given to mining hardware

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** template-derived search space given to mining hardware
- **Network target:** proof-of-work threshold for a valid Bitcoin block

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** template-derived search space given to mining hardware
- **Network target:** proof-of-work threshold for a valid Bitcoin block
- **Share target:** easier threshold used to measure a miner's work

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** template-derived search space given to mining hardware
- **Network target:** proof-of-work threshold for a valid Bitcoin block
- **Share target:** easier threshold used to measure a miner's work
- **Share:** valid proof of work for the pool, usually not a valid block

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** template-derived search space given to mining hardware
- **Network target:** proof-of-work threshold for a valid Bitcoin block
- **Share target:** easier threshold used to measure a miner's work
- **Share:** valid proof of work for the pool, usually not a valid block
- **Coinbase transaction:** first transaction; claims the subsidy and fees and carries the **extranonce**

Stratum V1



jobs flow right, shares flow left

V1 job flow

- Pool builds a block template and derives a mining job

V1 job flow

- Pool builds a block template and derives a mining job
- `mining.notify` sends the previous block hash, coinbase parts, Merkle branches, version, target bits, and time

V1 job flow

- Pool builds a block template and derives a mining job
- `mining.notify` sends the previous block hash, coinbase parts, Merkle branches, version, target bits, and time
- Miner varies the extranonce, nonce, time, and allowed version bits

V1 job flow

- Pool builds a block template and derives a mining job
- `mining.notify` sends the previous block hash, coinbase parts, Merkle branches, version, target bits, and time
- Miner varies the extranonce, nonce, time, and allowed version bits
- Miner returns hashes that meet the pool's easier share target

V1 job flow

- Pool builds a block template and derives a mining job
- `mining.notify` sends the previous block hash, coinbase parts, Merkle branches, version, target bits, and time
- Miner varies the extranonce, nonce, time, and allowed version bits
- Miner returns hashes that meet the pool's easier share target
- Pool validates the shares and pays for the measured work

Miner-controlled fields

- The extranonce changes the coinbase transaction

Miner-controlled fields

- The extranonce changes the coinbase transaction
- That changes the coinbase transaction ID and therefore the Merkle root

Miner-controlled fields

- The extranonce changes the coinbase transaction
- That changes the coinbase transaction ID and therefore the Merkle root
- The nonce, time, and permitted version bits provide more header search space

Miner-controlled fields

- The extranonce changes the coinbase transaction
- That changes the coinbase transaction ID and therefore the Merkle root
- The nonce, time, and permitted version bits provide more header search space
- These fields create many hashes from one pool-supplied job

Miner-controlled fields

- The extranonce changes the coinbase transaction
- That changes the coinbase transaction ID and therefore the Merkle root
- The nonce, time, and permitted version bits provide more header search space
- These fields create many hashes from one pool-supplied job
- None of them changes the non-coinbase transaction set

Pool controls the template

- ASICs dont have mempools

Pool controls the template

- ASICs don't have mempools
- The pool's node and template builder select the transactions

Pool controls the template

- ASICs don't have mempools
- The pool's node and template builder select the transactions
- A miner can point hash rate elsewhere, but cannot edit the current job

Pool controls the template

- ASICs don't have mempools
- The pool's node and template builder select the transactions
- A miner can point hash rate elsewhere, but cannot edit the current job
- A few pool operators therefore choose block contents for a large share of Bitcoin's hash rate

V1 limitations

- Pool-built templates
 - Pool chooses transactions for all connected hash rate

V1 limitations

- Pool-built templates
 - Pool chooses transactions for all connected hash rate
- JSON wire format
 - More bytes and parsing than a binary protocol needs

V1 limitations

- Pool-built templates
 - Pool chooses transactions for all connected hash rate
- JSON wire format
 - More bytes and parsing than a binary protocol needs
- Informal dialects and extensions
 - Compatibility depends on implementation quirks

V1 limitations

- Pool-built templates
 - Pool chooses transactions for all connected hash rate
- JSON wire format
 - More bytes and parsing than a binary protocol needs
- Informal dialects and extensions
 - Compatibility depends on implementation quirks
- No native authentication or encryption
 - Protection has to come from TLS, a VPN, or the network

V1 limitations

- Pool-built templates
 - Pool chooses transactions for all connected hash rate
- JSON wire format
 - More bytes and parsing than a binary protocol needs
- Informal dialects and extensions
 - Compatibility depends on implementation quirks
- No native authentication or encryption
 - Protection has to come from TLS, a VPN, or the network
- Coinbase and Merkle work on mining devices
 - More bandwidth and firmware complexity

Template Control

Split construction from accounting

Template construction

Transactions in the candidate block

Pool accounting

Accepted shares and miner payouts

Miner can build the template in both SV2 & DATUM (must in DATUM),
pool tracks shares and pays miners

- Miner's BTC node supplies the template

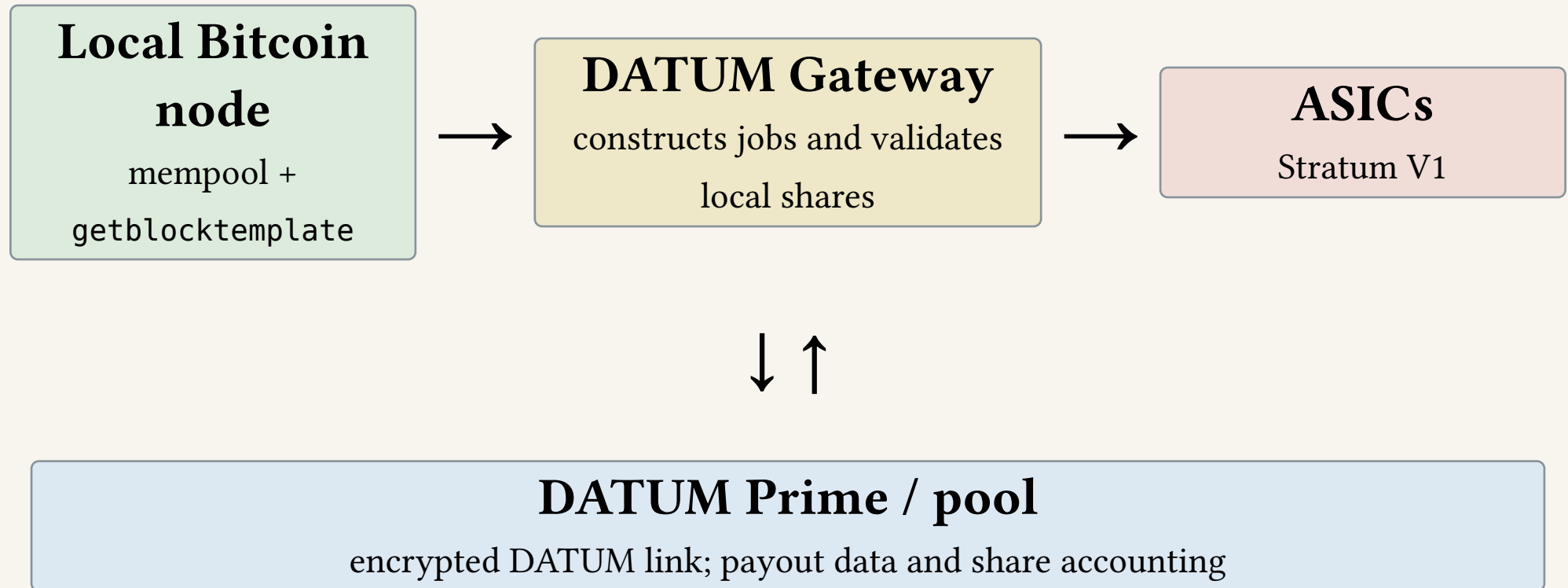
- Miner's BTC node supplies the template
- Its mempool and policy determine the transaction set

- Miner's BTC node supplies the template
- Its mempool and policy determine the transaction set
- Pool still accounts for shares and pays miners

- Miner's BTC node supplies the template
- Its mempool and policy determine the transaction set
- Pool still accounts for shares and pays miners
- Miner's node can publish a found block

DATUM

DATUM architecture



How DATUM works

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local node builds a template and returns it through `getblocktemplate`
- 2.
- 3.
- 4.
- 5.
- 6.

How DATUM works

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local node builds a template and returns it through `getblocktemplate`
2. The pool sends coinbase payout data and validity constraints
- 3.
- 4.
- 5.
- 6.

How DATUM works

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local node builds a template and returns it through `getblocktemplate`
2. The pool sends coinbase payout data and validity constraints
3. The Gateway builds the coinbase, Merkle root, and V1 jobs
- 4.
- 5.
- 6.

How DATUM works

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local node builds a template and returns it through `getblocktemplate`
2. The pool sends coinbase payout data and validity constraints
3. The Gateway builds the coinbase, Merkle root, and V1 jobs
4. ASICs hash those jobs and return shares to the Gateway
- 5.
- 6.

How DATUM works

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local node builds a template and returns it through `getblocktemplate`
2. The pool sends coinbase payout data and validity constraints
3. The Gateway builds the coinbase, Merkle root, and V1 jobs
4. ASICs hash those jobs and return shares to the Gateway
5. The Gateway checks shares and forwards qualifying ones to the pool
- 6.

How DATUM works

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local node builds a template and returns it through `getblocktemplate`
2. The pool sends coinbase payout data and validity constraints
3. The Gateway builds the coinbase, Merkle root, and V1 jobs
4. ASICs hash those jobs and return shares to the Gateway
5. The Gateway checks shares and forwards qualifying ones to the pool
6. The miner's node publishes a solved block

DATUM only mines local templates

- The Gateway creates work only from the miner's local node

DATUM only mines local templates

- The Gateway creates work only from the miner's local node
- DATUM defines no path for a pool-supplied template

DATUM only mines local templates

- The Gateway creates work only from the miner's local node
- DATUM defines no path for a pool-supplied template
- The pool cannot replace the local template without a configuration change

DATUM only mines local templates

- The Gateway creates work only from the miner's local node
- DATUM defines no path for a pool-supplied template
- The pool cannot replace the local template without a configuration change
- This guarantees local control but limits DATUM to that use case

Stratum V2

Three protocols

Protocol	Responsibility
Mining Protocol	Jobs, channels, targets, shares, and solutions
Template Distribution Protocol	Templates and transaction data from a full node
Job Declaration Protocol	Miner-created jobs proposed to a pool

The Mining Protocol can run alone. Job Declaration is optional, so SV2 does not automatically decentralize template construction.

SV2 roles

- **Template Provider:** node-side source of block templates

SV2 roles

- **Template Provider:** node-side source of block templates
- **Job Declarator Client (JDC):** miner-side coordinator for custom jobs

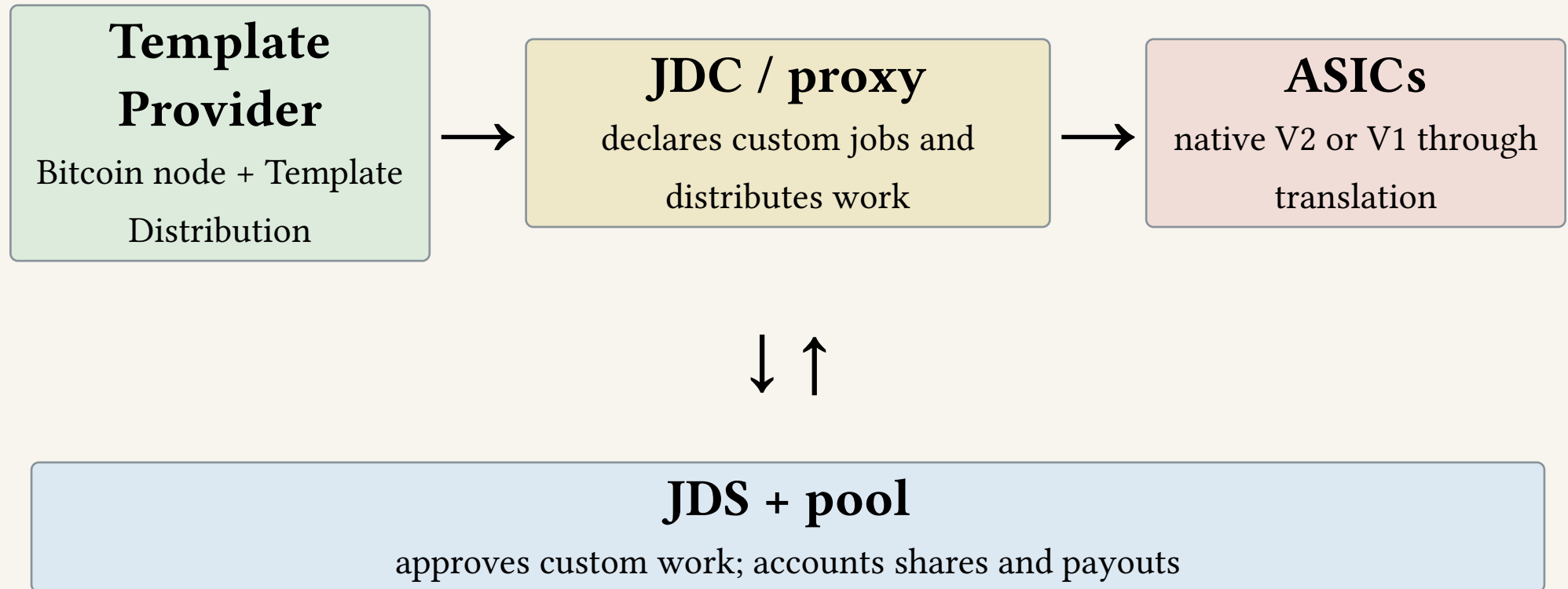
SV2 roles

- **Template Provider:** node-side source of block templates
- **Job Declarator Client (JDC):** miner-side coordinator for custom jobs
- **Job Declarator Server (JDS):** pool-side approval and validation service

SV2 roles

- **Template Provider:** node-side source of block templates
- **Job Declarator Client (JDC):** miner-side coordinator for custom jobs
- **Job Declarator Server (JDS):** pool-side approval and validation service
- **Mining proxy:** distributes jobs and aggregates downstream connections

Stratum V2 architecture



How a custom SV2 job works

1. Template Provider pushes a template to the miner-side JDC
- 2.
- 3.
- 4.
- 5.
- 6.

How a custom SV2 job works

1. Template Provider pushes a template to the miner-side JDC
2. JDC obtains a job token and required pool coinbase outputs
- 3.
- 4.
- 5.
- 6.

How a custom SV2 job works

1. Template Provider pushes a template to the miner-side JDC
2. JDC obtains a job token and required pool coinbase outputs
3. JDC builds a custom job from the local template
- 4.
- 5.
- 6.

How a custom SV2 job works

1. Template Provider pushes a template to the miner-side JDC
2. JDC obtains a job token and required pool coinbase outputs
3. JDC builds a custom job from the local template
4. Full-template reveals the transaction set; coinbase-only keeps it private
- 5.
- 6.

How a custom SV2 job works

1. Template Provider pushes a template to the miner-side JDC
2. JDC obtains a job token and required pool coinbase outputs
3. JDC builds a custom job from the local template
4. Full-template reveals the transaction set; coinbase-only keeps it private
5. The pool accepts the custom job and agrees to pay its shares
- 6.

How a custom SV2 job works

1. Template Provider pushes a template to the miner-side JDC
2. JDC obtains a job token and required pool coinbase outputs
3. JDC builds a custom job from the local template
4. Full-template reveals the transaction set; coinbase-only keeps it private
5. The pool accepts the custom job and agrees to pay its shares
6. The JDC distributes work and forwards shares; the miner's node publishes a found block

Two Job Declaration modes

Property	Full-template	Coinbase-only
Transaction set	Declared by transaction IDs; missing data can be requested	Not revealed to pool or JDS
Fee and validity checks	JDS can verify the proposed template	Pool cannot independently verify the transaction set
Privacy	JDS learns the template	Miner's mempool remains private
Tradeoff	More verification	More withholding / invalid- template risk for pool

SV2 transport and work distribution

- Binary framing with one precise specification

SV2 transport and work distribution

- Binary framing with one precise specification
- AEAD-encrypted remote connections with a Noise handshake

SV2 transport and work distribution

- Binary framing with one precise specification
- AEAD-encrypted remote connections with a Noise handshake
- Multiple channels per connection and efficient proxy aggregation

SV2 transport and work distribution

- Binary framing with one precise specification
- AEAD-encrypted remote connections with a Noise handshake
- Multiple channels per connection and efficient proxy aggregation
- Header-only jobs that keep coinbase and Merkle handling away from ASICs

SV2 transport and work distribution

- Binary framing with one precise specification
- AEAD-encrypted remote connections with a Noise handshake
- Multiple channels per connection and efficient proxy aggregation
- Header-only jobs that keep coinbase and Merkle handling away from ASICs
- Future jobs prepared before a new previous-block hash arrives

SV2 transport and work distribution

- Binary framing with one precise specification
- AEAD-encrypted remote connections with a Noise handshake
- Multiple channels per connection and efficient proxy aggregation
- Header-only jobs that keep coinbase and Merkle handling away from ASICs
- Future jobs prepared before a new previous-block hash arrives
- Native version rolling and specified protocol extensions

Comparison

SV2 and DATUM

Dimension	Stratum V2	DATUM
Scope	General mining protocol suite	Gateway for local templates and pool accounting
ASIC-facing link	Native V2, or translated V1	V1 today
Node-facing link	Template Distribution Protocol	getblocktemplate over RPC
Template source	Pool jobs or miner custom jobs	Miner's local node only
Template validation	Full-template or coinbase-only mode	Pool-coordinated today; intended to become mostly blind
Specification	Published modular protocol	Open implementation; pool protocol described as evolving
Remote security	Specified Noise/AEAD transport	Encrypted custom DATUM transport

Stratum V2 pros

- Replaces V1 on the pool link and inside the farm

Stratum V2 pros

- Replaces V1 on the pool link and inside the farm
- Gives pools, firmware, proxies, and nodes one published standard

Stratum V2 pros

- Replaces V1 on the pool link and inside the farm
- Gives pools, firmware, proxies, and nodes one published standard
- Uses the same security, framing, and channel model across implementations

Stratum V2 pros

- Replaces V1 on the pool link and inside the farm
- Gives pools, firmware, proxies, and nodes one published standard
- Uses the same security, framing, and channel model across implementations
- Defines a mining-specific path for node-to-miner templates

Stratum V2 pros

- Replaces V1 on the pool link and inside the farm
- Gives pools, firmware, proxies, and nodes one published standard
- Uses the same security, framing, and channel model across implementations
- Defines a mining-specific path for node-to-miner templates
- Supports verified full-template jobs and private coinbase-only jobs

Stratum V2 pros

- Replaces V1 on the pool link and inside the farm
- Gives pools, firmware, proxies, and nodes one published standard
- Uses the same security, framing, and channel model across implementations
- Defines a mining-specific path for node-to-miner templates
- Supports verified full-template jobs and private coinbase-only jobs
- Allows pool jobs and V1 translation during migration

DATUM's differences

- Existing V1 ASICs work unchanged behind one Gateway

DATUM's differences

- Existing V1 ASICs work unchanged behind one Gateway
- Local template construction is mandatory by design

DATUM's differences

- Existing V1 ASICs work unchanged behind one Gateway
- Local template construction is mandatory by design
- Its narrow design suits miners who only want local template control

DATUM's differences

- Existing V1 ASICs work unchanged behind one Gateway
- Local template construction is mandatory by design
- Its narrow design suits miners who only want local template control
- There is no pool-supplied fallback path

Comparison

Q&A